

Python E-Course

Module 12 — APIs & JSON

Detailed Notes

Module Overview

Level: Applied

Duration: ~1.5 weeks

Prerequisites: Modules 1-11

What You'll Learn

- Use HTTP fundamentals to call REST APIs.
- Parse JSON responses safely.
- Handle network failures, timeouts, and rate limits.
- Authenticate with API keys and tokens.
- Build a small wrapper around an external API.

Lessons

#	Lesson	Duration
1	HTTP Basics for Developers	30 min
2	Calling APIs with requests	35 min
3	Parsing JSON Responses	30 min
4	Authentication and Headers	30 min
5	Errors, Timeouts & Retries	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 13: Databases.

Lesson 1: HTTP Basics for Developers

Module: 12 — APIs & JSON

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Identify the parts of an HTTP request and response.
- Recognize common HTTP methods and status codes.
- Distinguish query parameters, path parameters, headers, and body.

Prerequisites

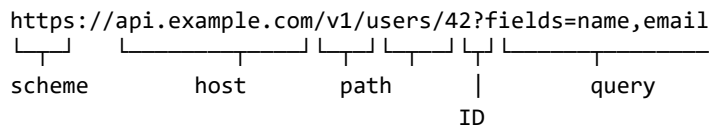
- Modules 1-11

1. Why This Matters

REST APIs are how programs talk to other programs over the web. Twitter, Stripe, OpenAI, your own backend — all speak HTTP. Five minutes of theory now will save hours of confusion when something returns a 401 or 422.

2. Concept

2.1 URL anatomy



- Scheme: http or https.
- Host: where to send the request.
- Path: identifies the resource.
- Query string: `?key=value&key2=value2`.

2.2 HTTP methods (verbs)

Method	Use
GET	Read data — no side effects
POST	Create — send a body
PUT	Replace
PATCH	Partial update
DELETE	Remove

2.3 Status codes

Range	Meaning
1xx	Informational (rare)
2xx	Success (200 OK, 201 Created, 204 No

	Content)
3xx	Redirect (301, 302, 304)
4xx	Client error (400, 401, 403, 404, 422, 429)
5xx	Server error (500, 502, 503, 504)

Memorize: 200, 201, 204, 301, 400, 401, 403, 404, 422, 429, 500, 503. The rest you can look up.

2.4 Headers

Key/value metadata about the request or response:

```
Authorization: Bearer abc123
Content-Type: application/json
Accept: application/json
User-Agent: my-app/1.0
```

2.5 Body

Used by POST/PUT/PATCH. Often JSON in modern APIs:

```
{"name": "Maya", "email": "m@x.com"}
```

2.6 Query vs body

- Query: filters, paging, options. GET /users?limit=10&page=2.
- Body: new or updated resource data. POST /users with {name: ..., email: ...}.

2.7 REST conventions

- GET /users — list users.
- GET /users/42 — fetch user 42.
- POST /users — create.
- PUT /users/42 — replace user 42.
- DELETE /users/42 — remove user 42.

Most APIs follow this; some don't. Always read the docs.

3. Code Examples

(For this lesson, no Python yet — just inspecting URLs and understanding the pieces.)

Example 1: Parse a URL with `urllib.parse`

```
from urllib.parse import urlparse, parse_qs
url = "https://api.example.com/v1/users/42?fields=name,email&limit=10"
parsed = urlparse(url)
print(parsed.scheme)
print(parsed.netloc)
print(parsed.path)
print(parse_qs(parsed.query))
```

Expected Output:

```
https
api.example.com
```

```
/v1/users/42
{'fields': ['name,email'], 'limit': ['10']}
```

Example 2: Build a URL with query params

```
from urllib.parse import urlencode
base = "https://api.example.com/search"
params = {"q": "python tutorials", "limit": 20}
url = base + "?" + urlencode(params)
print(url)
```

Expected Output: `https://api.example.com/search?q=python+tutorials&limit=20`

Example 3: Read a sample response

```
HTTP/1.1 200 OK
Content-Type: application/json
X-RateLimit-Remaining: 59
```

```
{"id": 42, "name": "Maya"}
```

Identify: status (200), headers (Content-Type, X-RateLimit-Remaining), body (JSON).

4. Common Mistakes

Mistake	What Happens	Fix
Confusing 401 with 403	401 = not logged in; 403 = logged in but not allowed	Check the auth
Sending body on GET	Most APIs ignore it; some break	Use query params
Forgetting Content-Type: application/json	Server can't parse	Set the header
Treating 2xx as the only "ok"	204 returns no body	Handle no-content responses

5. Practice Tasks

- Identify the parts of `https://api.weather.com/v1/forecast?city=Pune&days=5`.
- Which method would you use to create a new user? Which to fetch one?
- Look up what HTTP 429 means.

6. Key Takeaways

- HTTP = method + URL + headers + body.
- Status codes: 2xx success, 4xx your fault, 5xx server's fault.
- REST conventions: GET to read, POST to create, PUT/PATCH to update, DELETE to remove.

7. What's Next

We'll make our first real HTTP request with the requests library.

Lesson 2: Calling APIs with requests

Module: 12 — APIs & JSON

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Install and use the requests library.
- Make GET, POST, PUT, DELETE calls.
- Pass query parameters and JSON bodies cleanly.

Prerequisites

- Module 10 venv/pip, Module 12 Lesson 1.

1. Why This Matters

requests is the de facto Python HTTP library. It's not in the stdlib, but you'll see it in almost every Python codebase. Its API is so good it shaped how many other languages do HTTP.

2. Concept

2.1 Install

In your active venv:

```
pip install requests
```

2.2 Basic GET

```
import requests
r = requests.get("https://httpbin.org/get")
print(r.status_code)
print(r.text)           # body as string
print(r.json())         # parsed JSON (if Content-Type is JSON)
```

2.3 Query params

Pass a dict — no manual encoding:

```
r = requests.get("https://httpbin.org/get",
                 params={"city": "Pune", "days": 5})
print(r.url)  # shows the encoded URL
```

2.4 POST with JSON body

```
payload = {"name": "Maya", "email": "m@x.com"}
r = requests.post("https://httpbin.org/post", json=payload)
print(r.json())
```

json= automatically encodes and sets Content-Type: application/json.

For form data, use data= instead.

2.5 PUT, PATCH, DELETE

```
requests.put(url, json=payload)
requests.patch(url, json=payload)
requests.delete(url)
```

Same arguments as get/post.

2.6 Inspecting the response

Attribute	Meaning
r.status_code	Int
r.text	Body as string
r.content	Body as bytes
r.json()	Body parsed as JSON (raises if invalid)
r.headers	Dict of response headers
r.url	Final URL (after redirects)
r.ok	True if 200-399
r.raise_for_status()	Raises HTTPError if 4xx/5xx

3. Code Examples

Example 1: Simple GET

```
import requests
r = requests.get("https://httpbin.org/get",
                 params={"q": "python"})
print(r.status_code)
print(r.url)
print(r.json()["args"])
```

Expected Output:

```
200
https://httpbin.org/get?q=python
{'q': 'python'}
```

Example 2: POST JSON

```
import requests
r = requests.post(
    "https://httpbin.org/post",
    json={"name": "Maya", "city": "Pune"},
)
data = r.json()
print(data["json"])
```

Expected Output: {'name': 'Maya', 'city': 'Pune'}

Example 3: Reading headers

```
import requests
r = requests.get("https://httpbin.org/get")
print(r.headers["Content-Type"])
print(r.headers.get("X-Missing", "n/a"))
```

Expected Output:

```
application/json  
n/a
```

Example 4: raise_for_status

```
import requests  
r = requests.get("https://httpbin.org/status/404")  
try:  
    r.raise_for_status()  
except requests.HTTPError as e:  
    print("error:", e)
```

Expected Output: error: 404 Client Error: NOT FOUND for url: https://httpbin.org/status/404

Example 5: Real public API — joke

```
import requests  
r = requests.get("https://official-joke-api.appspot.com/random_joke")  
joke = r.json()  
print(joke["setup"])  
print(joke["punchline"])
```

Expected Output: (a random joke)

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting to check status code	Errors silently	Use <code>r.raise_for_status()</code>
Building query strings by hand	Encoding bugs	Use <code>params=</code>
Sending dict as <code>data=</code> for JSON APIs	Treated as form-encoded	Use <code>json=</code>
Not setting a User-Agent	Some APIs reject	<code>headers={"User-Agent": "my-app/0.1"}</code>

5. Practice Tasks

- GET `https://httpbin.org/get?city=Pune` and print the parsed JSON.
- POST `{"x": 1}` to `https://httpbin.org/post` and print the echoed json field.
- Use `raise_for_status` on a `httpbin.org/status/500` request and catch the error.

6. Key Takeaways

- `requests.get/post/put/patch/delete` cover REST.
- `params=` for query, `json=` for JSON body, `headers=` for headers.
- `r.json()` to parse; `r.raise_for_status()` to assert success.

7. What's Next

A deeper dive on parsing JSON — drilling into nested fields safely.

Lesson 3: Parsing JSON Responses

Module: 12 — APIs & JSON

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Navigate nested JSON safely.
- Use dict.get chains and helper functions for defensive parsing.
- Convert API responses into Python dataclasses.

Prerequisites

- Module 5 Lesson 7, Module 9 Lesson 7, Module 12 Lesson 2.

1. Why This Matters

API responses are JSON. JSON parses into Python dicts and lists — and reading deeply nested structures is a daily skill. Doing it defensively (not crashing on missing fields) is what separates fragile scripts from production code.

2. Concept

2.1 JSON → Python

Done by `response.json()` or `json.loads()`. Result is a dict (object) or list (array).

2.2 Direct lookup with []

```
data = {"user": {"name": "Maya"}}
print(data["user"]["name"])  # "Maya"
print(data["user"]["age"])    # KeyError
```

Fast but brittle if fields can be missing.

2.3 Defensive lookup with .get

```
data["user"].get("age")          # None
data["user"].get("age", "unknown") # "unknown"
data.get("posts", [])           # default for missing list
```

For deep chains:

```
city = data.get("user", {}).get("address", {}).get("city")
```

2.4 Helper function for paths

```
def dig(obj, *path, default=None):
    for key in path:
        if isinstance(obj, dict):
            obj = obj.get(key)
        elif isinstance(obj, list) and isinstance(key, int):
            obj = obj[key] if 0 <= key < len(obj) else None
        else:
            return default
    return obj
```

```

        else:
            return default
        if obj is None:
            return default
    return obj

```

Use:

```
dig(data, "user", "address", "city", default="n/a")
```

2.5 Mapping to dataclasses

For consistency, often you want a typed Python object:

```

from dataclasses import dataclass

@dataclass
class User:
    id: int
    name: str
    email: str | None = None

def make_user(raw):
    return User(id=raw["id"], name=raw["name"], email=raw.get("email"))

```

Library options like pydantic do this with validation — useful in larger projects.

2.6 Iterating over a list response

```

for item in data["items"]:
    print(item["title"])

```

Most list endpoints return {"items": [...], "total": N} or similar; some return the bare list.

2.7 Type checking

```

if isinstance(value, list):
    ...

```

Useful when an API may return either a single object or a list.

3. Code Examples

Example 1: Nested lookup

```

data = {
    "user": {
        "name": "Maya",
        "address": {"city": "Pune", "pin": 411001},
    }
}
print(data["user"]["address"]["city"])
print(data["user"].get("phone", "no phone"))

```

Expected Output:

```
Pune
no phone
```

Example 2: Safe deep get

```
def dig(obj, *path, default=None):
    for key in path:
        if isinstance(obj, dict):
            obj = obj.get(key)
        elif isinstance(obj, list) and isinstance(key, int):
            obj = obj[key] if 0 <= key < len(obj) else None
        else:
            return default
    if obj is None:
        return default
    return obj

data = {"a": {"b": [{"c": "deep"}]}}
print(dig(data, "a", "b", 0, "c"))
print(dig(data, "a", "x", "y", default="missing"))
```

Expected Output:

```
deep
missing
```

Example 3: List of items

```
import requests
r = requests.get("https://jsonplaceholder.typicode.com/users")
users = r.json()
for u in users[:3]:
    print(u["name"], "-", u["company"]["name"])
```

Expected Output (sample):

```
Leanne Graham - Romaguera-Crona
Ervin Howell - Deckow-Crist
Clementine Bauch - Romaguera-Jacobson
```

Example 4: Dataclass mapping

```
from dataclasses import dataclass

@dataclass
class User:
    id: int
    name: str
    email: str | None = None

raw = {"id": 1, "name": "Maya", "email": "m@x.com"}
u = User(**{k: v for k, v in raw.items() if k in {"id", "name", "email"}})
print(u)
```

Expected Output: User(id=1, name='Maya', email='m@x.com')

Explanation: Filter unknown keys before unpacking so extra fields don't crash.

Example 5: Pretty-printing for inspection

```
import json
data = {"a": 1, "b": [2, 3, {"c": "deep"}]}
print(json.dumps(data, indent=2))
```

Expected Output:

```
{
  "a": 1,
  "b": [
    2,
    3,
    {
      "c": "deep"
    }
  ]
}
```

4. Common Mistakes

Mistake	What Happens	Fix
Direct chain d[a][b][c] on flaky data	KeyError	Use .get chain or dig
Treating list and dict interchangeably	TypeError	Check isinstance
Trusting fields will always be present	Random failures	Provide defaults
Unpacking dict with unknown keys into dataclass	TypeError: unexpected keyword argument	Filter to known keys first

5. Practice Tasks

- Given {"a": {"b": {"c": 1}}}, fetch the deep value with .get chain.
- Write a dig helper and test on a nested fixture.
- Convert {"id": 5, "name": "Test", "extra": "ignored"} into a User(id, name) dataclass.

6. Key Takeaways

- API responses become dicts/lists in Python.
- Use .get with defaults to avoid KeyError.
- Build small dig helpers for deep paths.
- Map raw dicts into dataclasses for clarity.

7. What's Next

Adding authentication — most real APIs require an API key or token.

Lesson 4: Authentication and Headers

Module: 12 — APIs & JSON

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Add API keys and tokens to requests securely.
- Use environment variables to keep secrets out of code.
- Use Session for persistent headers and connection reuse.

Prerequisites

- Module 12 Lesson 3

1. Why This Matters

Real APIs require authentication. Putting your secret API key in your source code (or worse, committing it to git) is a leak waiting to happen. This lesson teaches the right way to handle credentials.

2. Concept

2.1 Common auth schemes

Scheme	How
API key in header	Authorization: Bearer ABC123 or custom X-API-Key: ABC123
API key in query	?api_key=ABC123 (older / less secure)
Basic auth	Authorization: Basic base64(user:pass)
OAuth / JWT	Bearer token after a separate login flow

The docs of the API tell you which.

2.2 Passing a header in requests

```
headers = {"Authorization": "Bearer abc123"}
r = requests.get("https://api.example.com/me", headers=headers)
```

2.3 Basic auth shortcut

```
r = requests.get(url, auth=("username", "password"))
```

requests builds the Authorization header for you.

2.4 Loading secrets from environment variables

Never hard-code secrets:

```
import os
API_KEY = os.environ["API_KEY"] # raises if not set
# or
API_KEY = os.environ.get("API_KEY", "")
```

Set the variable before running:

Windows (PowerShell):

```
$env:API_KEY = "abc123"
python my_script.py
```

macOS/Linux:

```
export API_KEY=abc123
python my_script.py
```

2.5 .env files (optional)

For convenience, use a .env file + python-dotenv:

```
# .env
API_KEY=abc123

from dotenv import load_dotenv
load_dotenv()
API_KEY = os.environ["API_KEY"]
```

Don't commit .env — add it to .gitignore.

2.6 Session for persistent headers

A Session reuses TCP connections and shared headers:

```
import requests
s = requests.Session()
s.headers.update({"Authorization": "Bearer abc"})
r1 = s.get("https://api.example.com/me")
r2 = s.get("https://api.example.com/orders")
```

Use a session whenever you make multiple calls to the same host.

2.7 Don't print secrets in logs

When logging requests, mask the key:

```
print(headers["Authorization"][:10] + "...")
```

3. Code Examples

Example 1: Header auth (illustrative — replace API_KEY)

```
import os
import requests

API_KEY = os.environ.get("DEMO_KEY", "missing")
headers = {"Authorization": f"Bearer {API_KEY}"}
r = requests.get("https://httpbin.org/headers", headers=headers)
print(r.json()["headers"]["Authorization"])
```

Expected Output: Bearer missing (or your actual key)

Example 2: Basic auth

```
import requests
r = requests.get("https://httpbin.org/basic-auth/user/passwd",
                 auth=("user", "passwd"))
print(r.status_code, r.json())
```

Expected Output:

```
200 {'authenticated': True, 'user': 'user'}
```

Example 3: Session

```
import requests
s = requests.Session()
s.headers.update({"X-App": "demo/0.1"})
for path in ["/get", "/get?x=1", "/get?x=2"]:
    r = s.get("https://httpbin.org" + path)
    print(path, "->", r.status_code)
```

Expected Output:

```
/get -> 200
/get?x=1 -> 200
/get?x=2 -> 200
```

Example 4: Loading from env

```
import os
api_key = os.environ.get("API_KEY")
if not api_key:
    raise SystemExit("Please set API_KEY in your environment.")
print("key starts with:", api_key[:4])
```

Expected Output: key starts with: abc1 (or whatever)

Example 5: Mask in logs

```
def mask(token):
    return token[:4] + "..." + token[-2:] if len(token) > 6 else "****"
print(mask("sk_test_abcdef1234"))
```

Expected Output: sk_t...34

4. Common Mistakes

Mistake	What Happens	Fix
Hard-coding API key in source	Leak in commits	Use env vars or .env
Committing .env to git	Same leak	.gitignore it
Reusing one global header dict across hosts	Wrong header sent	Per-host headers or per-call
Logging full token	Token in logs	Mask first 4 / last 2 only

5. Practice Tasks

- Make an authenticated GET `https://httpbin.org/bearer` with `Authorization: Bearer test-token`.
- Read an env var `MY_KEY` and use it as a header value. Pass missing case gracefully.
- Create a Session with `User-Agent: course/0.1` and make two requests.

6. Key Takeaways

- API keys live in headers (usually `Authorization: Bearer ...`).
- Load secrets from environment variables, not source.
- Use Session for multiple calls to the same API.
- Mask tokens in logs.

7. What's Next

Errors, timeouts, and retries — how to make API calls robust on a flaky network.

Lesson 5: Errors, Timeouts & Retries

Module: 12 — APIs & JSON

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Always set a request timeout.
- Handle network exceptions and HTTP errors distinctly.
- Implement simple retry-with-backoff for transient failures.
- Respect rate limits.

Prerequisites

- Module 8, Module 12 Lesson 4.

1. Why This Matters

The network fails. Servers go down. Rate limits kick in. A script that crashes on the first hiccup is useless in production. This lesson teaches the small set of defenses that make HTTP code reliable.

2. Concept

2.1 Always set a timeout

By default, requests waits forever. Always pass `timeout=`:

```
r = requests.get(url, timeout=5)
```

A tuple (connect, read) gives finer control:

```
r = requests.get(url, timeout=(3, 10))
```

Without a timeout, your program can hang for hours.

2.2 Distinct exception types

Exception	When
<code>requests.exceptions.Timeout</code>	Request took too long
<code>requests.exceptions.ConnectionError</code>	DNS failure, refused, dropped
<code>requests.exceptions.HTTPError</code>	Raised by <code>raise_for_status()</code> on 4xx/5xx
<code>requests.exceptions.RequestException</code>	Parent of all of the above

Catch the narrowest you can react to.

2.3 Pattern: try + raise_for_status

```
import requests
from requests.exceptions import RequestException

try:
```

```

r = requests.get(url, timeout=5)
r.raise_for_status()
data = r.json()
except RequestException as e:
    print("API call failed:", e)
    data = None

```

2.4 Retries with backoff

Transient errors (502, 503, 504, connection resets) often succeed if retried with a small delay:

```

import time

def get_with_retry(url, attempts=3, backoff=1.0):
    for i in range(attempts):
        try:
            r = requests.get(url, timeout=5)
            if r.status_code < 500:
                return r
        except requests.RequestException:
            pass
        time.sleep(backoff * (2 ** i)) # exponential
    raise RuntimeError(f"all {attempts} attempts failed")

```

Don't retry on 4xx errors (your fault) — only on 5xx and network errors.

2.5 Rate limits

A 429 Too Many Requests response means "slow down." Check for a Retry-After header and wait:

```

if r.status_code == 429:
    delay = int(r.headers.get("Retry-After", 5))
    time.sleep(delay)

```

For batch jobs, throttle proactively (e.g., `time.sleep(0.1)` between calls).

2.6 Idempotency

Retries are safe only for idempotent operations — calling twice has the same effect as once. GET and DELETE are usually idempotent; POST often is not (you'd create two records). Use an idempotency key if the API supports one.

2.7 requests.adapters (advanced — quick mention)

For production, library code often uses `HTTPAdapter + urllib3.Retry` to centralize retry policy. Worth knowing about; not required here.

3. Code Examples

Example 1: Timeout

```

import requests
try:
    r = requests.get("https://httpbin.org/delay/3", timeout=1)

```

```
except requests.exceptions.Timeout:
    print("timed out")
```

Expected Output: timed out

Example 2: Distinguish errors

```
import requests
from requests.exceptions import HTTPError, Timeout, RequestException

def safe_get(url):
    try:
        r = requests.get(url, timeout=3)
        r.raise_for_status()
        return r.json()
    except Timeout:
        return {"error": "timeout"}
    except HTTPError as e:
        return {"error": f"http {e.response.status_code}"}
    except RequestException as e:
        return {"error": f"network {e}"}

print(safe_get("https://httpbin.org/status/404"))
```

Expected Output: {'error': 'http 404'}

Example 3: Retry with backoff

```
import time, requests

def get_with_retry(url, attempts=3):
    for i in range(attempts):
        try:
            r = requests.get(url, timeout=5)
            if r.status_code < 500:
                return r
            print(f"attempt {i+1}: status {r.status_code}, retrying")
        except requests.RequestException as e:
            print(f"attempt {i+1}: {e}, retrying")
            time.sleep(2 ** i)
    raise RuntimeError(f"gave up after {attempts} attempts")

# This URL returns 500 randomly; for demo purposes you can replace with /status/500
# to see backoff
try:
    r = get_with_retry("https://httpbin.org/status/500", attempts=2)
except RuntimeError as e:
    print(e)
```

Expected Output (sample):

```
attempt 1: status 500, retrying
attempt 2: status 500, retrying
gave up after 2 attempts
```

Example 4: Respect Retry-After

```
import time, requests

def call(url):
    r = requests.get(url, timeout=5)
    if r.status_code == 429:
        wait = int(r.headers.get("Retry-After", 1))
        print(f"rate-limited; sleeping {wait}s")
        time.sleep(wait)
        r = requests.get(url, timeout=5)
    return r

# httpbin doesn't have a true 429-with-Retry-After endpoint, but the pattern is
# what matters.
```

Expected Output: (depends on endpoint)

4. Common Mistakes

Mistake	What Happens	Fix
No timeout=	Script hangs forever	Always set timeout
Retrying on 4xx	Same error every time	Only retry on 5xx / network
Retrying POST blindly	Duplicate writes	Make idempotent (or use idempotency keys)
Hammering on 429	IP gets banned	Honor Retry-After; back off

5. Practice Tasks

- Add a timeout=2 to any requests.get you've written.
- Wrap a call in try/except, catching Timeout and HTTPError separately.
- Write a tiny retry(func, n=3) decorator that retries on RequestException.

6. Key Takeaways

- Always set timeout=.
- Distinguish Timeout, HTTPError, ConnectionError.
- Retry only on transient (5xx, network) failures, with backoff.
- Respect Retry-After.

7. What's Next

End of Module 12. Next: Module 13 — Databases, where data persists.

Module 12 — Exercises

15 exercises.

12.1 HTTP basics

- (E) Parse `https://api.x.com/v1/items?limit=10` with `urllib.parse`.
- (M) Use `urlencode` to build a URL with `{"q": "python", "limit": 5}`.
- (H) Write a function that classifies a status code as "success", "client error", or "server error".

12.2 requests

- (E) GET `https://httpbin.org/get`. Print status and `r.json()`.
- (M) POST `{"x": 1}` JSON to `https://httpbin.org/post`. Print the echoed json field.
- (H) GET `https://httpbin.org/status/404`; use `raise_for_status` and catch `HTTPError`.

12.3 JSON

- (E) Given `{"a": {"b": {"c": 1}}}`, fetch 1 with `.get`.
- (M) Write a `dig(obj, *path)` helper and test it.
- (H) Map a `{"id": 1, "name": "x", "extra": "y"}` into a `User(id, name)` dataclass, filtering unknown keys.

12.4 Auth

- (E) Send Authorization: Bearer test123 to `https://httpbin.org/headers`; print the echoed header.
- (M) Read `API_KEY` from `os.environ`. Print a masked version.
- (H) Use `Session` to GET 3 paths with a shared User-Agent header.

12.5 Errors/timeouts/retries

- (E) Set `timeout=2` on a GET to `https://httpbin.org/delay/5`. Catch `Timeout`.
- (M) Wrap a call in `try/except` with distinct branches for `Timeout` and `HTTPError`.
- (H) Write a `retry(times=3)` decorator that retries on `RequestException` with `time.sleep(2**i)` backoff.

Module 12 — Quiz

10 MCQs.

Q1

Which method is used to fetch a resource without modifying it? A. POST B. PUT C. GET D. DELETE

Answer: C (L1)

Q2

Which range is "server error"? A. 2xx B. 3xx C. 4xx D. 5xx

Answer: D (L1)

Q3

In requests, which arg sends a JSON body? A. data= B. json= C. body= D. payload=

Answer: B (L2)

Q4

Which gives parsed JSON from a response? A. r.json() B. r.text C. r.content D. json(r)

Answer: A (L2)

Q5

Which call raises on 4xx/5xx? A. r.raise_for_status() B. r.assert_ok() C. r.check() D. r.ensure()

Answer: A (L2)

Q6

What's the safest way to fetch a deep nested value? A. d[a][b][c] B. d.get(a, {}).get(b, {}).get(c) C. d[a][b].get(c) D. d.deep(a, b, c)

Answer: B (L3)

Q7

Where should an API key go? A. Hard-coded in the script B. Committed to git in a constants file C. Read from os.environ D. Stored in a string

Answer: C (L4)

Q8

What does Session give you? A. Cookies + persistent headers + connection reuse B. Cleaner URLs C. Automatic retries D. JSON helpers

Answer: A (L4)

Q9

What's wrong with `requests.get(url)`? A. Nothing B. Missing method C. Missing timeout= D. Returns synchronously

Answer: C — always set a timeout. (L5)

Q10

You should retry on: A. Any 4xx B. 401 Unauthorized C. 5xx and network errors D. All errors

Answer: C (L5)

Module 12 — Assignment: Public API Client

Estimated time: 2 hours

Combines: Module 12 + Modules 4, 7, 9-11

Brief

Build a small Python client for a free public API. Recommended: the JSONPlaceholder API (no auth needed) at <https://jsonplaceholder.typicode.com>. Wrap a few endpoints in a tidy client class.

Requirements

Implement class `JsonPlaceholderClient`:

- `__init__(self, base_url="https://jsonplaceholder.typicode.com", timeout=5)` — creates a `requests.Session`, stores base URL and timeout.
- `_get(self, path, params=None)` — helper that builds the URL, sends the request with the configured timeout, calls `raise_for_status`, returns `r.json()`.
- `list_users(self)` → list of `User` dataclasses.
- `get_user(self, user_id)` → `User` (or raises).
- `list_posts(self, user_id=None)` → list of `Post` dataclasses (filter by `user_id` if given).
- `comments_for_post(self, post_id)` → list of `Comment` dataclasses.
- Dataclasses: `User(id, name, email)`, `Post(id, user_id, title, body)`, `Comment(id, post_id, name, email, body)`.

A `main()` that:

- Lists the first 3 users.
- For user 1, prints their first 2 posts.
- For the first post, prints the count of comments.
- All wrapped in `try/except` for `RequestException`.

Constraints

- Use a `Session`.
- Always pass `timeout=`.
- Use `.json()` and dataclasses; do NOT just print raw dicts.
- Define a custom `ApiError(Exception)` raised when the client wraps a `RequestException`.

Expected Output (sample)

```
== Users (first 3) ==
1. Leanne Graham <Sincere@april.biz>
2. Ervin Howell <Shanna@melissa.tv>
3. Clementine Bauch <Nathan@yesenia.net>

== Posts by user 1 (first 2) ==
```


- sunt aut facere repellat provident occaecati excepturi optio reprehenderit
- qui est esse

== Comments on post 1 ==
5 comments

Grading Rubric

Criterion	Weight
Client class structure	25%
Dataclasses used	15%
All endpoints work	25%
Error handling	20%
Style + docstrings	15%

Submission

Save as assignment_module12.py. Include requirements.txt listing requests.

Module 12 — Mini Project: GitHub User Card

Overview

Build a CLI tool that takes a GitHub username and prints a "card" with name, bio, follower count, public repo count, and top 3 most-starred repos. Uses GitHub's public API (no auth required, but rate-limited).

Concepts Used

- requests (M12 L2)
- JSON parsing (M12 L3)
- Headers + sessions (M12 L4)
- Timeouts + error handling (M12 L5)
- Dataclasses (M9)

Features

- Accept username via CLI arg (`sys.argv[1]`) or interactive prompt.
- Fetch `/users/{name}` and `/users/{name}/repos`.
- Sort repos by `stargazers_count`; show top 3.
- Set User-Agent header (required by GitHub).
- Handle missing user (404) gracefully.

Starter Code

github_card.py:

```
import sys
import requests
from dataclasses import dataclass
from requests.exceptions import RequestException

BASE = "https://api.github.com"
SESSION = requests.Session()
SESSION.headers.update({"User-Agent": "py-course-card/0.1"})

@dataclass
class Profile:
    name: str
    bio: str
    followers: int
    public_repos: int

@dataclass
class Repo:
    name: str
    stars: int
    description: str

def get_profile(username):
```

```

    r = SESSION.get(f"{BASE}/users/{username}", timeout=5)
    if r.status_code == 404:
        raise LookupError(f"user '{username}' not found")
    r.raise_for_status()
    d = r.json()
    return Profile(
        name=d.get("name") or username,
        bio=d.get("bio") or "(no bio)",
        followers=d.get("followers", 0),
        public_repos=d.get("public_repos", 0),
    )

def get_top_repos(username, n=3):
    r = SESSION.get(
        f"{BASE}/users/{username}/repos",
        params={"per_page": 100, "sort": "updated"},
        timeout=5,
    )
    r.raise_for_status()
    repos = [
        Repo(name=x["name"], stars=x.get("stargazers_count", 0),
            description=x.get("description") or "")
        for x in r.json()
    ]
    return sorted(repos, key=lambda r: r.stars, reverse=True)[:n]

def render_card(profile, repos):
    print("=" * 50)
    print(f" {profile.name}")
    print(f" {profile.bio}")
    print("-" * 50)
    print(f" Followers : {profile.followers}")
    print(f" Repos      : {profile.public_repos}")
    print(" Top repos :")
    for r in repos:
        print(f"      * {r.name}  (★{r.stars})")
        if r.description:
            print(f"      {r.description[:60]}")
    print("=" * 50)

def main():
    user = sys.argv[1] if len(sys.argv) > 1 else input("GitHub username: ").strip()
    try:
        profile = get_profile(user)
        repos = get_top_repos(user)
        render_card(profile, repos)
    except LookupError as e:
        print(e)
    except RequestException as e:
        print("network error:", e)

if __name__ == "__main__":
    main()

```

Expected Interaction

```
$ python github_card.py torvalds
=====
Linux Torvalds
(no bio)
-----
Followers : 200000
Repos     : 8
Top repos :
  * linux (★180000)
    Linux kernel source tree
  * subsurface (★2900)
  * test-tlb (★400)
=====
```

Extension Challenges

- Add --save FILE to dump the result as JSON.
- Add support for organizations (/orgs/{name}).
- Cache responses for 10 minutes in a local JSON file.

Grading Rubric

Criterion	Weight
Works for real user	25%
404 handled gracefully	15%
Uses Session + UA	15%
Dataclasses used	15%
Top-N sorting correct	15%
Style + docstrings	15%